KHÓA THỰC HÀNH CURSOR • BUỔI 4 / 5 • 90 phút

Bắt đầu làm theo nhóm



Mỗi nhóm 3–4 người sẽ cùng làm một ứng dụng.
Các thay đổi code được chia sẻ bằng PR.

0:00 • cả lớp

Trước khi bắt đầu

Trước khi vào bài, cả lớp cùng kiểm tra 4 việc sau.

- [ ] Xem bảng chia nhóm để biết mình ở nhóm nào và số nhóm
- [ ] Đăng nhập được GitHub (chương 3 sẽ tạo repository)
- [ ] **Dùng được GitHub CLI.** Chạy `gh auth status` trong terminal (`Ctrl+``) và thấy `Logged in`
- [ ] Hôm nay không làm việc bên trong `cursor-course/`

Ai chưa có `gh` thì cài ngay từ `https://cli.github.com/`, khởi động lại Cursor rồi chạy `gh auth login`. Lệnh này cần cho `/start-task` và cho việc tạo PR, Issue.

NỘI DUNG HÔM NAY

Nội dung thực hành hôm nay

Sử dụng các file yêu cầu, task và rule (Quy tắc) đã tạo ở buổi trước, hôm nay **cả nhóm** sẽ cùng làm một ứng dụng.

	Việc cần làm	Chương
1	Tạo một repository chung cho cả nhóm	Chương 3
2	Thêm rule đã tạo ở buổi trước vào repository để làm **rule của nhóm**	Chương 4
3	**Cả nhóm** viết yêu cầu và task, rồi chuyển task thành Issue	Chương 5
4	Phân chia task và bắt đầu triển khai	Chương 6

Từ hôm nay, thay đổi code được chia sẻ bằng **PR (pull request)**. **Không cần hoàn thành ứng dụng.** Việc hoàn thiện và trình bày sẽ làm ở buổi 5.

LỘ TRÌNH 90 PHÚT

Thời gian biểu

Giờ	Chương	Nội dung	Phụ trách
0:00	Chương 1 Mục tiêu hôm nay	Giải thích cách tiến hành, phân chia vai trò (5 phút)	GV
0:05	Chương 2 Chọn chủ đề	Chọn chủ đề trong danh sách (5 phút)	Nhóm
0:10	Chương 3 Tạo repository	Tạo từ template → mời thành viên → mọi người clone (12 phút)	Nhóm
0:22	**Chương 4 Tạo PR đầu tiên**	**Thêm file rule bằng PR (18 phút)**	Nhóm
0:40	Chương 5 Tạo PR yêu cầu và task	Yêu cầu → task → PR → chuyển task thành Issue (17 phút)	Nhóm
0:57	Chương 6 Làm task 1 và phân chia task	/start-task cho task 1 → merge → mỗi người /start-task (23 phút)	Nhóm
1:20	Chương 7 Tổng kết	Báo cáo tiến độ, giới thiệu buổi sau (10 phút)	GV

> Thời gian học viên tự thực hành là **75 / 90 phút**. **Chương 4 không rút ngắn dù bị trễ giờ**, vì đây là chương duy nhất cả lớp cùng xem các bước làm PR.

GÓC MỞ RỘNG • Thuật ngữ cơ bản của GitHub

Thuật ngữ sử dụng từ hôm nay

Vì đây là lần đầu dùng GitHub nên giải thích thuật ngữ trước. Học viên không cần nhớ cơ chế chi tiết.

Từ	Nghĩa	Tình huống dùng hôm nay
GitHub	Dịch vụ web để đặt repository lên Internet và chia sẻ trong nhóm	Đặt repository của nhóm
Repository (repo)	Tập hợp các file của một ứng dụng và lịch sử thay đổi của chúng	Cả nhóm tạo một cái
Branch	Nơi thực hiện thay đổi tách riêng khỏi main. Sửa trên branch thì main không thay đổi	Mỗi task tạo một branch
main	Branch gốc của repository. Chỉ đưa vào những gì nhóm đã kiểm tra	Dùng để trình bày ở buổi 5
Commit	Ghi lại thay đổi thành một mốc	Mỗi khi hoàn tất một việc
PR (pull request)	Lời đề nghị “Hãy kiểm tra xem thay đổi của branch này đưa vào main được chưa”	Khi chia sẻ thay đổi
Merge	Đưa thay đổi của PR vào main	Sau khi kiểm tra xong

CHƯƠNG 1 • Vai trò trong nhóm

Vai trò trong nhóm và quy tắc PR

Với nhóm 3 người, Tech Lead kiêm nhiệm vai trò QA.

Vai trò	Số người	Công việc hôm nay
Tech Lead	1	Tạo repository / mời thành viên / **tạo PR đầu tiên (file rule)**
PM	1	Dùng `/requirements` và `/task-breakdown` để làm yêu cầu và task, rồi **tạo PR** / quyết định khi ý kiến khác nhau / người nói chính khi trình bày ở buổi 5
Engineer	1	**Triển khai task 1 và tạo PR**
QA	1	Chạy thử trước thao tác sẽ trình bày / **chạy PR trên trình duyệt để kiểm tra, rồi merge** / thao tác màn hình khi trình bày ở buổi 5

Quy tắc PR: ① Không tự merge PR của mình ② Không commit thẳng vào branch `main`.
Khi không có việc của mình, hãy đọc và kiểm tra PR của người khác.

CHƯƠNG 2 • Cả nhóm (5 phút)

Chọn chủ đề

Hãy chọn 1 trong 5 chủ đề (3 phút), làm như công việc do "khách hàng" trong bảng đặt. **Có thể trùng chủ đề với nhóm khác.**
Nếu muốn chọn chủ đề ngoài danh sách, hãy hỏi giảng viên. **Ở buổi 5, nhóm sẽ thực hiện thao tác ở cột bên phải.**

#	Chủ đề	Khách hàng	Thao tác trình bày
1	Gọi món trên điện thoại	Quán cà phê	Khách đặt món trên màn hình khách thì món hiện ở màn hình bếp; bấm "Đã phục vụ" thì màn hình khách đổi trạng thái
2	Đặt ghế rạp chiếu phim	Rạp chiếu phim	Chọn 2 ghế và loại vé thì hiện tổng tiền; đặt xong thì không chọn được ghế đó nữa
3	Bốc thăm khuyến mãi	Phòng marketing hãng nước giải khát	Bấm nút thì trúng quà và số quà còn lại giảm; quà đã hết thì không trúng nữa
4	Trắc nghiệm gợi ý sản phẩm	Chuỗi cà phê	Trả lời 5 câu hỏi thì hiện kiểu người và loại cà phê được gợi ý
5	Máy bán hàng tự động	Hãng máy bán hàng	Bỏ 20.000 đồng vào rồi mua một món giá 12.000 đồng thì ra hàng và hiện chi tiết tiền thối

CHƯƠNG 2 • Điểm cần xác nhận theo từng chủ đề

Những điều cần xác nhận với khách hàng

Nếu không ghi vào yêu cầu, AI sẽ tự quyết định. Phần **in đậm** là các mục đặc biệt quan trọng.

Chủ đề	Cần quyết định trong yêu cầu	Ví dụ "không làm gì"
Gọi món	Tuỳ chọn (size, topping) và giá / **món đã hết** / huỷ đơn đến lúc nào / trạng thái đơn	Thanh toán / đăng nhập / nhiều cửa hàng
Đặt ghế	Loại vé và giá / mỗi lần chọn tối đa mấy ghế / **có cho chọn để lại đúng 1 ghế trống lẻ không** / ghế xe lăn	Thanh toán / nhiều suất chiếu / thành viên
Bốc thăm	**Tỉ lệ trúng của từng quà** / quà hết thì tỉ lệ đó xử lý thế nào / mỗi ngày bốc mấy lần	Gửi quà / đăng nhập / hiệu ứng cầu kỳ
Trắc nghiệm	Điểm của câu hỏi và lựa chọn / **kết quả khi bằng điểm** / có quay lại câu trước được không	Đăng mạng xã hội / tạo ảnh / lưu câu trả lời
Máy bán hàng	Loại tiền nhận / **khi không đủ tiền lẻ để thối** / hết hàng / trả lại tiền	Ví điện tử / hiển thị nhiệt độ / thống kê doanh thu

GÓC MỞ RỘNG • Thuật ngữ khi tạo repository

Thuật ngữ khi tạo repository

Từ	Nghĩa	Tình huống dùng hôm nay
Template repository	Repository dùng làm mẫu khi tạo repository mới. Nhấn Use this template để tạo repository có sẵn các file giống vậy	Tạo từ template do giảng viên chuẩn bị
Collaborator	Thành viên được mời vào với quyền ghi vào repository	Tech Lead mời thành viên
Clone	Sao chép repository trên GitHub về máy của mình	Mọi người đều thực hiện
GitHub CLI (gh)	Công cụ để thao tác GitHub bằng lệnh trong terminal	Khi Agent tạo PR / Issue và khi dùng `/start-task`

CHƯƠNG 3 • Tech Lead (5 phút)

Tạo repository và mời thành viên

1 Use this template → Create a new repository

Template: `https://github.com/xrnd-tec/cursor-team-template`
Tên repo: `team-<số>-<tên-app>` (ví dụ `team-3-cinema`)

2 Settings → Collaborators → Add people

Nhập tên người dùng GitHub của thành viên. Thêm **tất cả** thành viên

3 Thành viên khác: Accept invitation

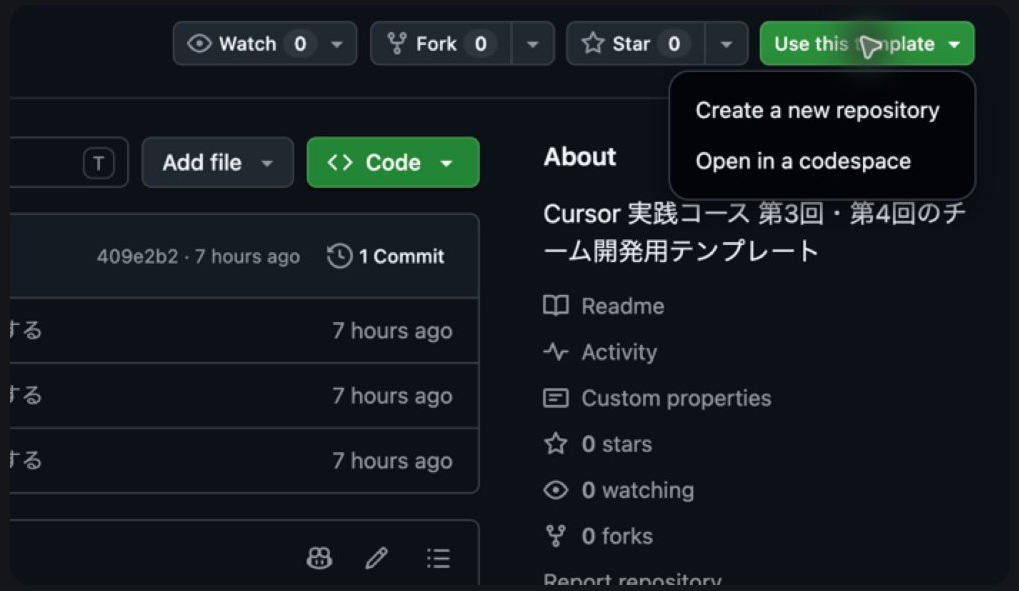
Chấp nhận lời mời từ email GitHub gửi đến hoặc từ thông báo trên GitHub

Nếu không thấy email, hãy mở `https://github.com/<trưởng-nhóm>/<tên-repo>/invitations`.

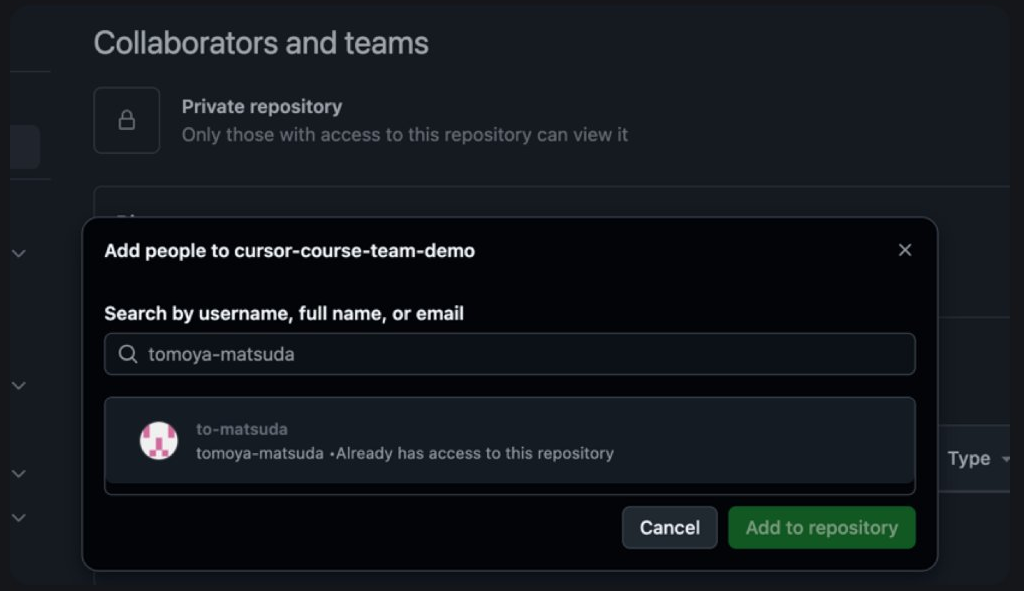
CHƯƠNG 3 • Màn hình

Màn hình tạo repository và màn hình mời

Use this template → Create a new repository



Settings → Collaborators → Add people



> Khi mời, nhập **tên người dùng GitHub** của thành viên rồi nhấn **Add to repository**. Hãy thêm **tất cả** thành viên.

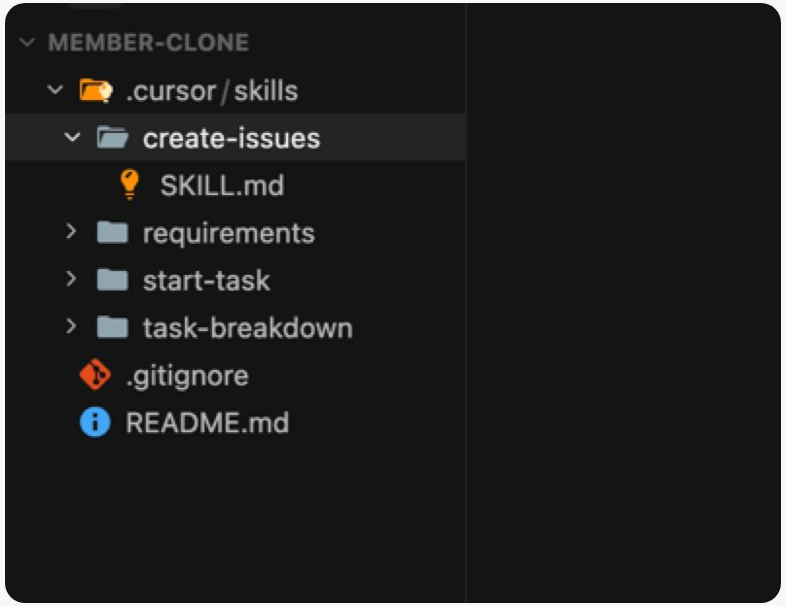
CHƯƠNG 3 • Mọi người (5 phút)

Clone repository và kiểm tra thiết lập

`Ctrl+Shift+P` (Mac: `Cmd+Shift+P`) → `Git: Clone` → dán URL → lưu **bên ngoài** `cursor-course/` → Open.
Khi đã thấy `.cursor/skills/` ở thanh bên, hãy gửi đoạn dưới cho Agent.

```
Kiểm tra xem git trên máy này đã đặt user.name và user.email chưa.
Kiểm tra luôn gh auth status xem đã đăng nhập GitHub CLI chưa.
Nếu thiếu gì thì chỉ cho tôi biết cần làm gì. Đừng tự cài đặt.
```

> Nếu thiếu user.name / user.email, hãy cho Agent biết tên và email GitHub để Agent thiết lập giúp.



GÓC MỞ RỘNG • Thuật ngữ khi chia sẻ thay đổi

Thuật ngữ khi chia sẻ thay đổi

Từ	Nghĩa	Tên trên màn hình
Stage	Chọn file sẽ đưa vào commit	Nút + trong bảng Source Control
Push	Gửi branch và commit trên máy mình lên GitHub	**Publish Branch** (lần đầu) / **Sync Changes**
Pull	Lấy thay đổi mới trên GitHub về máy mình	**Sync Changes**
Review / Approve	Kiểm tra thay đổi của PR / kiểm tra xong và xác nhận "không có vấn đề"	**Files changed** / **Review changes → Approve**
Conflict	Trạng thái 2 người sửa cùng một chỗ trong một file nên Git không tự gộp được	This branch has conflicts

CHƯƠNG 4 • Các bước làm PR

Các bước làm PR

Bước 1–5 do người tạo PR, bước 6 do người kiểm tra PR, bước 7 do mọi người thực hiện.

1. Tạo branch (tên branch ở góc dưới bên trái Cursor)
2. Sửa file (bằng Agent)
3. Commit (Source Control → + → ✨ → Commit)
4. Gửi lên GitHub (Publish Branch)
5. Tạo PR (yêu cầu Agent / trên màn hình GitHub)
6. Thành viên khác kiểm tra, Approve → merge (trên màn hình GitHub)
7. Mọi người cập nhật main (chuyển sang main → Sync Changes)

CHƯƠNG 4 • Bảng Source Control

Các nút dùng trong bảng Source Control

Hãy mở “Source Control” ở thanh bên trái (`Ctrl+Shift+G`, trên Mac cũng giống vậy).

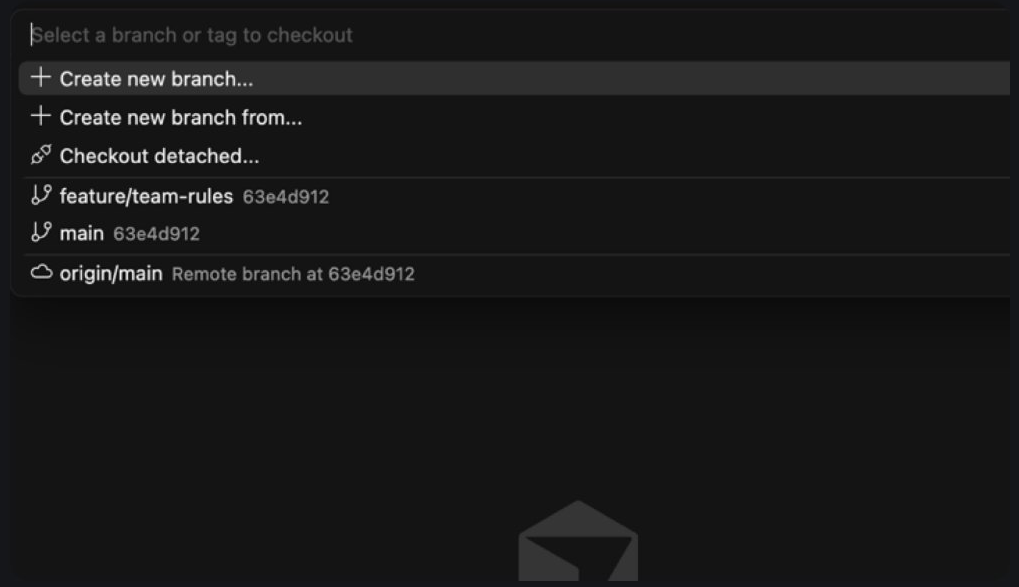
Vị trí	Chức năng
Tên branch ở góc dưới bên trái (thanh trạng thái)	Nhấp → **Create new branch...** để tạo branch / chuyển branch
Nút + ở Changes	Chọn file sẽ commit (stage)
Nút ✨ ở ô nhập	AI viết commit message. **Kiểm tra nội dung rồi mới** Commit
Publish Branch / Sync Changes	Gửi lên GitHub / lấy từ GitHub về

Commit message do ✨ tạo cần **kiểm tra nội dung rồi mới** commit.
Khi nhờ Agent tạo PR, Agent dùng `gh pr create`. Nếu không có `gh`, hãy tạo trên màn hình GitHub bằng **Compare & pull request**.

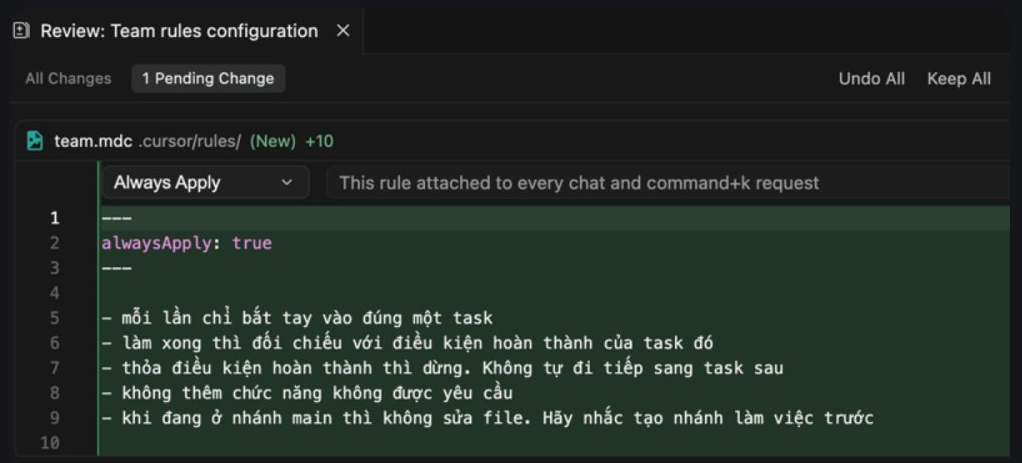
CHƯƠNG 4 • Giảng viên làm mẫu

Tạo branch và yêu cầu Agent tạo file rule

Tên branch ở góc dưới bên trái → Create new branch...



Xác nhận diff chỉ thêm đúng 5 dòng rồi nhấn Keep

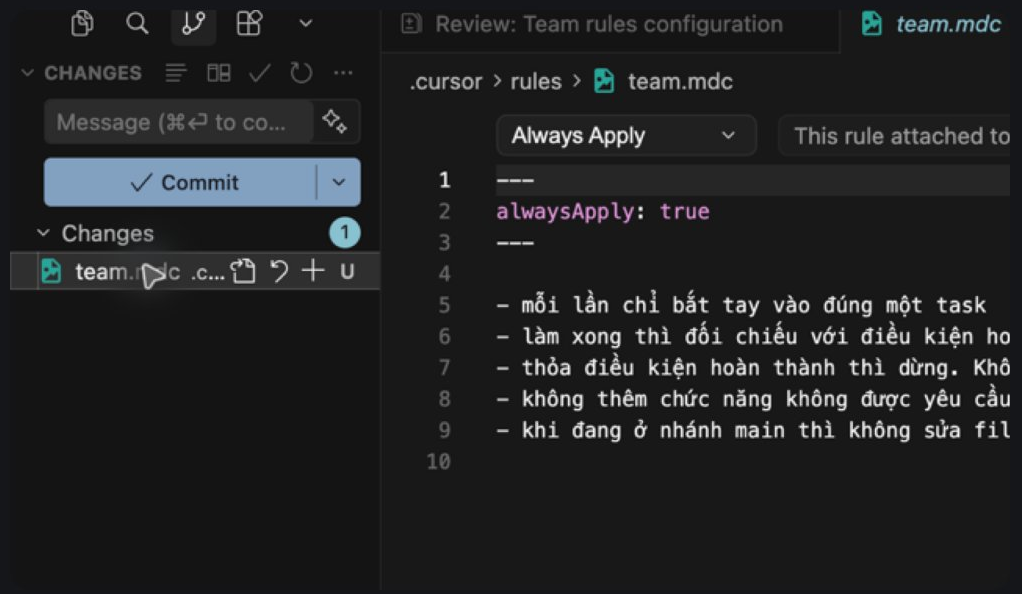


> Tên branch là `feature/team-rules`. Với file rule do Agent tạo, hãy xem diff có **đúng 5 dòng đã nhờ** không rồi mới nhấn Keep.

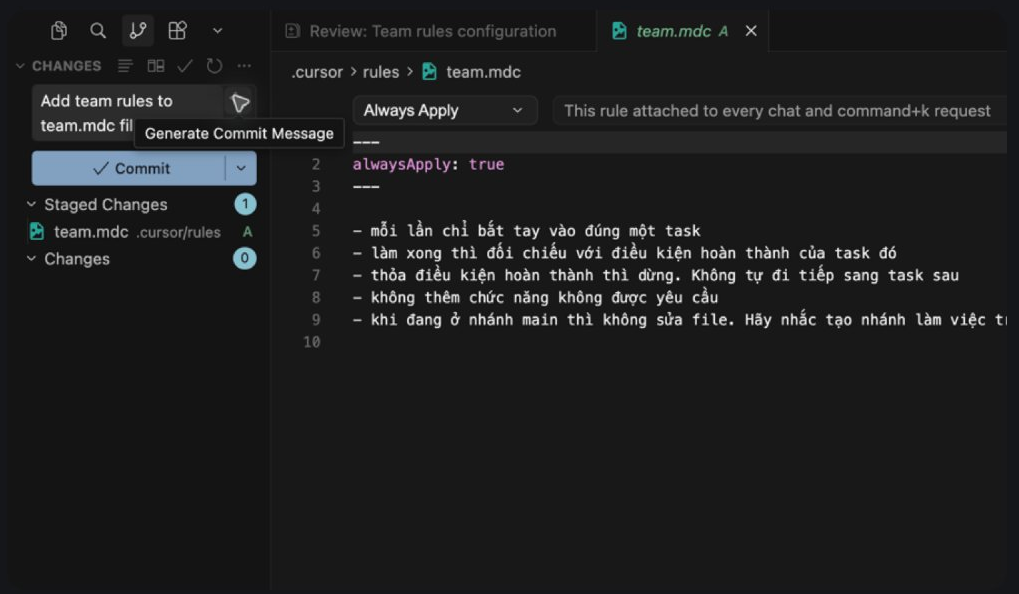
CHƯƠNG 4 • Giảng viên làm mẫu ②

Stage rồi commit

Nhấn + ở team.mdc trong Changes



Tạo message bằng ✨, kiểm tra rồi Commit

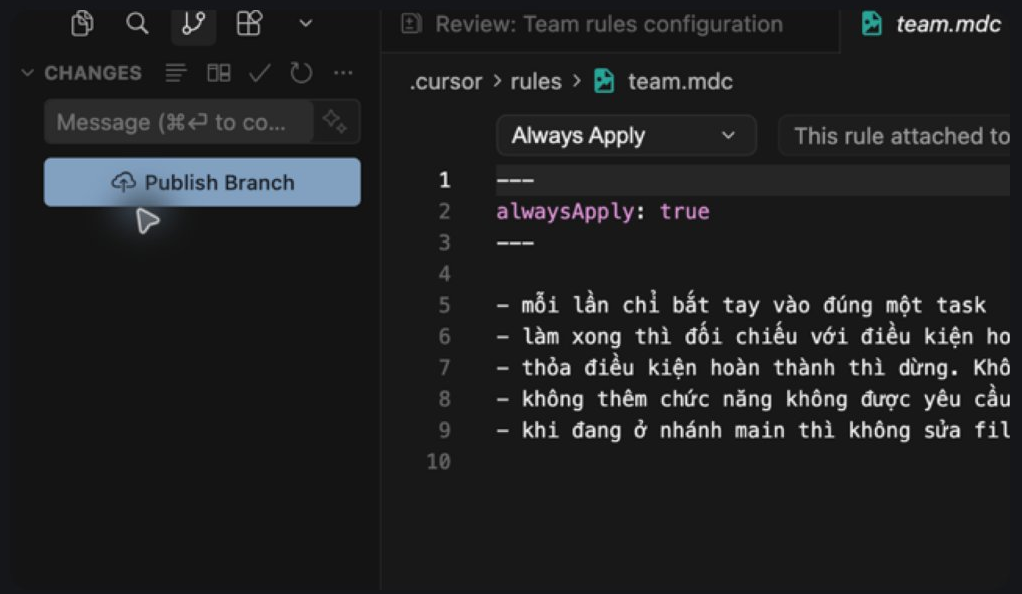


> Khi nhấn +, file sẽ chuyển sang **Staged Changes**. Message tạo bằng ✨ cần được **kiểm tra nội dung** rồi mới Commit.

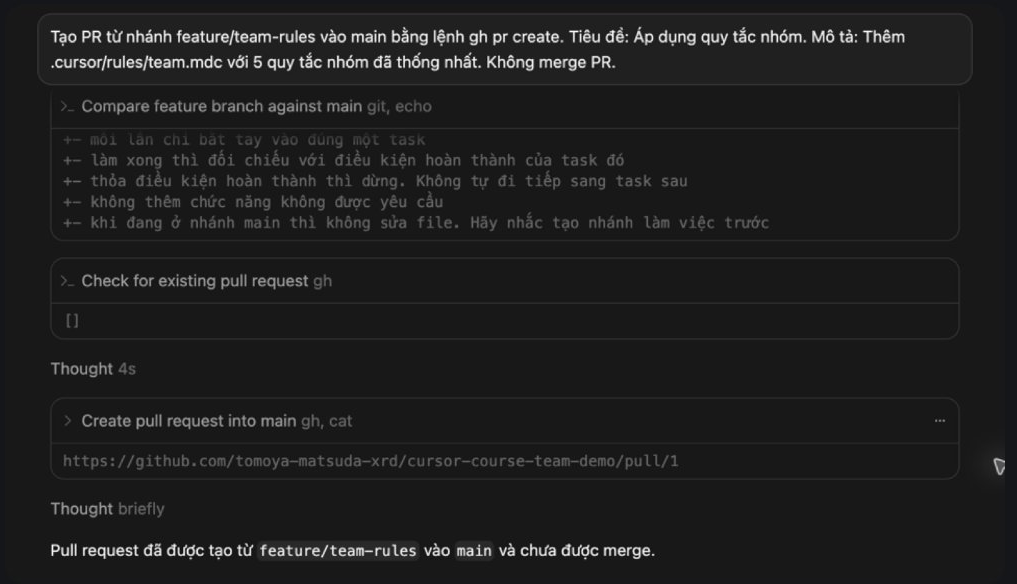
CHƯƠNG 4 • Giảng viên làm mẫu ⑤

Gửi lên GitHub và tạo PR

Gửi lên GitHub bằng Publish Branch



Nhờ Agent tạo PR

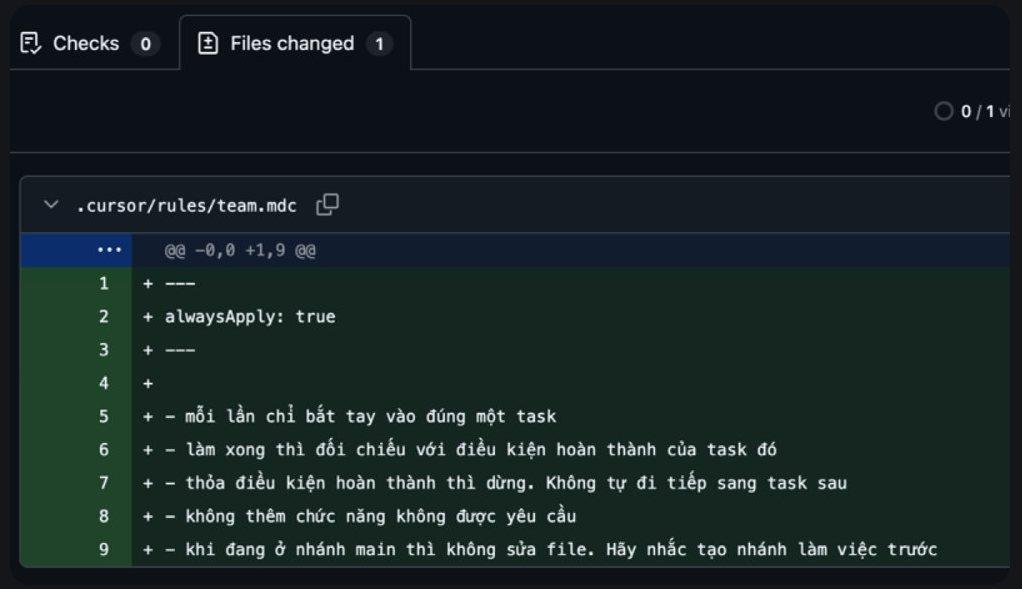


> Khi nhờ Agent, PR được tạo bằng `gh pr create` và URL của PR sẽ hiện ra. Nếu không có `gh`, hãy tạo bằng **Compare & pull request** trên màn hình GitHub.

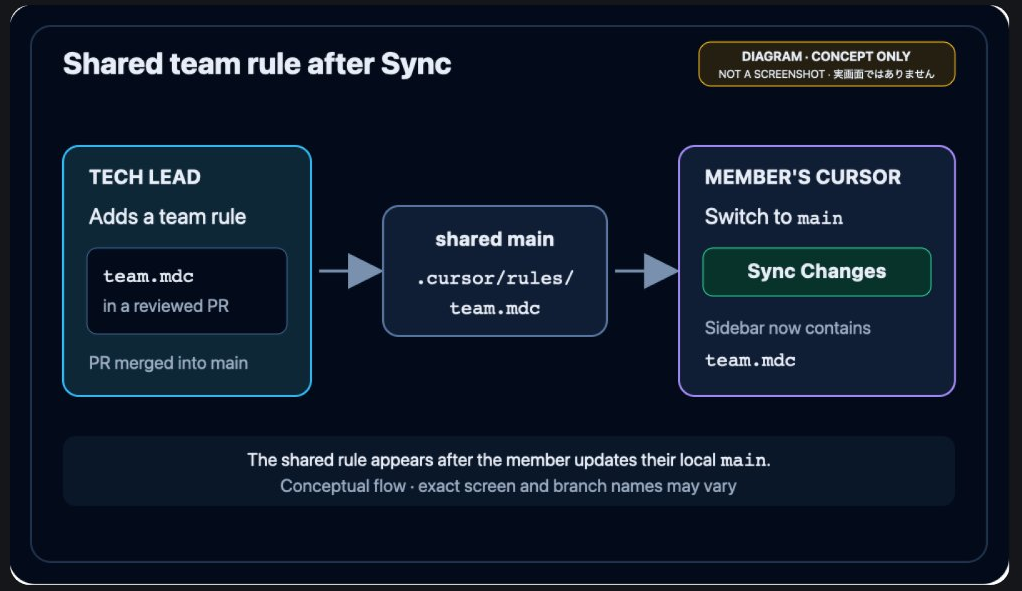
CHƯƠNG 4 • Giảng viên làm mẫu

Kiểm tra, merge rồi cập nhật về máy của mọi người

Xem Files changed rồi Approve



Sau khi thành viên khác nhấn Sync



> Người kiểm tra xác nhận chỉ có **đúng 5 dòng đã nhờ** được thêm vào. Sau khi merge, khi thành viên khác nhấn Sync Changes ở main thì file rule sẽ hiện ra.

CHƯƠNG 4 • Tech Lead (2 phút)

Tạo file rule

Trong chat mới, hãy gửi đoạn dưới. **Nội dung gồm 4 dòng của buổi 3 và thêm 1 dòng cho nhóm.**

```
Tạo file .cursor/rules/team.mdc.
Đặt alwaysApply: true.
Nội dung chỉ gồm đúng 5 dòng sau. Đừng thêm gì khác.

- mỗi lần chỉ bắt tay vào đúng một task
- làm xong thì đối chiếu với điều kiện hoàn thành của task đó
- thỏa điều kiện hoàn thành thì dừng. Không tự đi tiếp sang task sau
- không thêm chức năng không được yêu cầu
- khi đang ở branch main thì không sửa file. Hãy nhắc tạo branch làm việc trước
```

Dòng thứ 5 là rule để **AI dừng lại khi bắt đầu làm trên branch `main`**.

CHƯƠNG 4 • Tech Lead (7 phút)

Các bước đến khi tạo PR

1 Tạo branch (1 phút)

Tên branch ở góc dưới bên trái → Create new branch... → `feature/team-rules`

2 Yêu cầu Agent tạo file rule (2 phút)

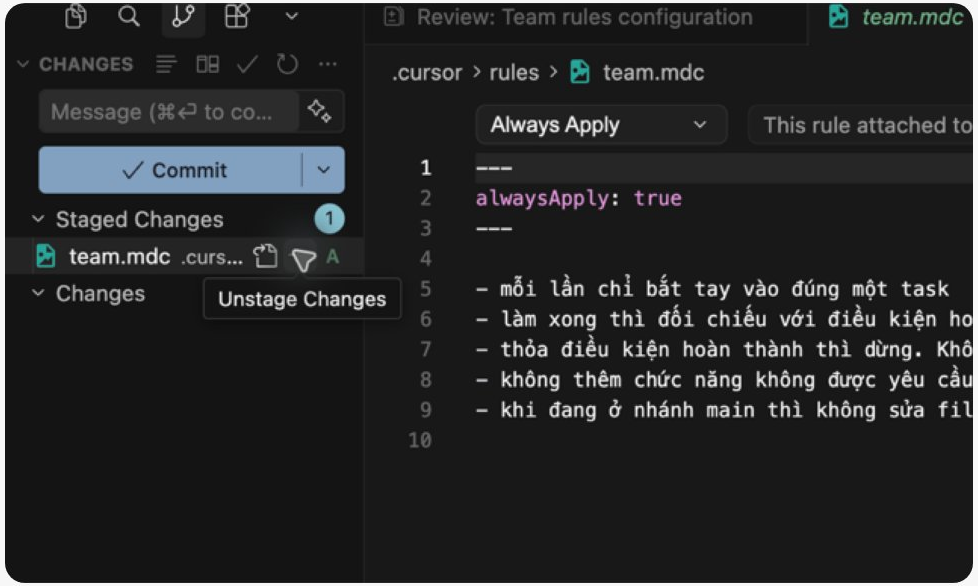
Gửi đoạn ở trang trước. Xem diff, nếu **chỉ thêm 5 dòng** thì nhấn Keep

3 Commit và gửi lên GitHub (2 phút)

Source Control → + ở `team.mdc` → ✨ → đọc lại → Commit → Publish Branch

4 Tạo PR (2 phút)

Nếu có `gh` thì nhờ Agent. Nếu không có thì Compare & pull request → Create pull request



CHƯƠNG 4 • 1 thành viên ngoài Tech Lead (3 phút)

Kiểm tra PR rồi merge

1. **Pull requests → mở PR của Tech Lead**

   Mở repository của nhóm trên GitHub

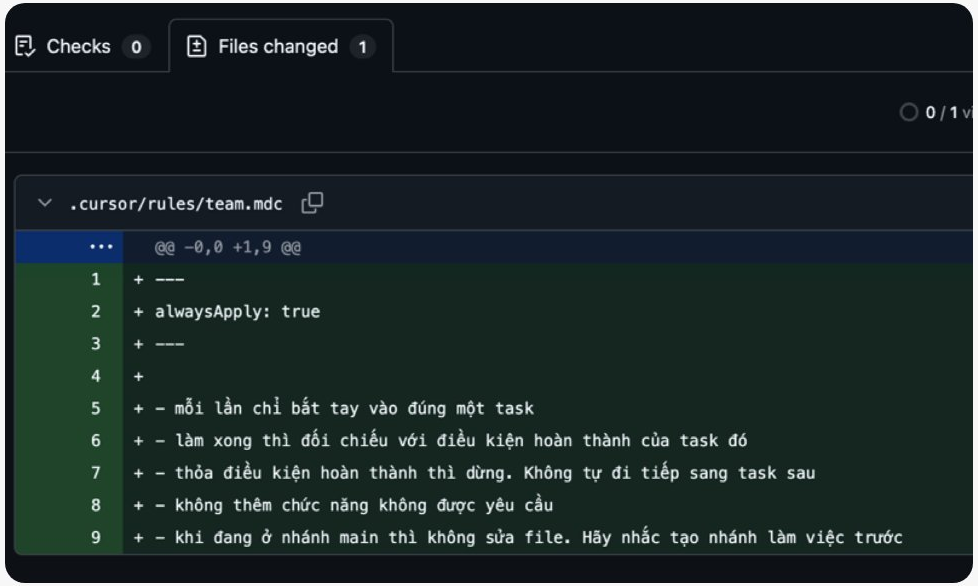
2. **Xem 5 dòng ở Files changed**

   Kiểm tra **có dòng nào không được nhờ mà vẫn bị thêm không**

3. **Review changes → Approve → Submit review**

   Không Approve được PR của chính mình

4. **Merge pull request → Confirm merge**



CHƯƠNG 4 • Mọi người (3 phút)

Cập nhật main

1. **Bấm tên branch ở góc dưới trái → chọn `main`**

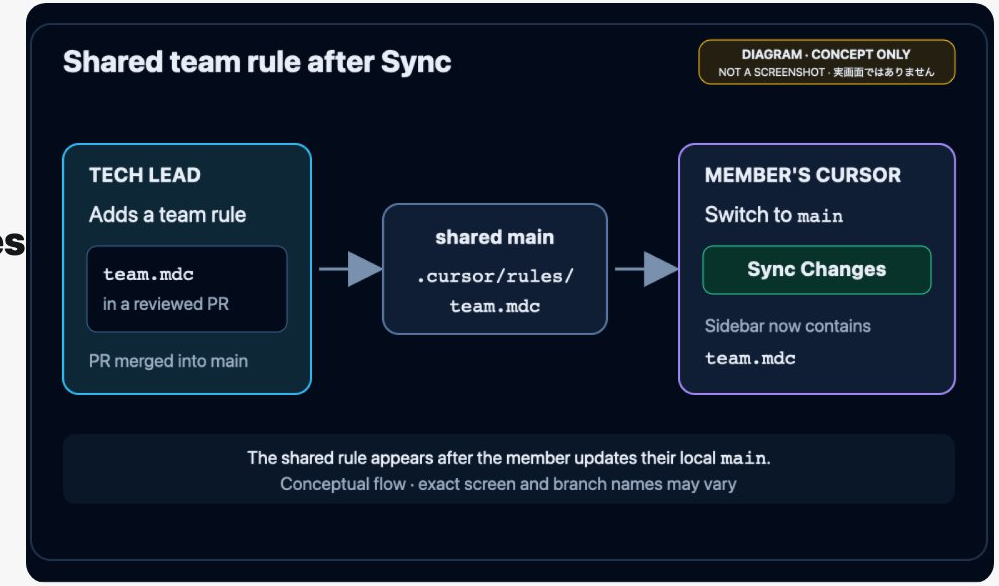
   Bấm vào tên branch, rồi chọn `main` trong danh sách. Kiểm tra góc dưới trái đã đổi thành `main`

2. **Mở biểu tượng Source Control ở bên trái, nhấn Sync Changes**

   Nếu không thấy nút này thì dùng `Ctrl+Shift+P` → `Git: Pull`

3. **Kiểm tra có `.cursor/rules/team.mdc` chưa**

   Xem ở thanh bên



File rule do Tech Lead thêm vào đã có trên máy của mọi người. Ở buổi 3, rule chỉ được tạo trên máy của mình, còn lần này cả nhóm dùng chung một rule trong Cursor.

CHƯƠNG 5 • Cách chia task

Cách chia task khi làm nhóm

Khác với buổi 3, yêu cầu do **PM viết và cả nhóm cùng thảo luận**, rồi mọi người kiểm tra bằng PR. Nếu không ghi "không làm gì", mỗi người sẽ tự thêm chức năng khác nhau.

Quy định	Lý do
Task 1 làm đến khi "màn hình hiện ra"	Các task khác được làm trên nền task 1. **Task 1 chưa merge thì chưa bắt đầu được task khác**
Số task ≥ số người	Mỗi người phụ trách ít nhất 1 task
1 task = 1 branch = 1 PR	PR càng nhỏ càng dễ kiểm tra
Quản lý người phụ trách và trạng thái hoàn thành bằng Issue	Nếu ghi vào tasks.md, cả nhóm sẽ sửa cùng một file và PR bị xung đột. **Bắt đầu bằng** /start-task**, Issue đóng là hoàn tất**

CHƯƠNG 5 • PM (cả nhóm cùng trả lời câu hỏi)

Viết yêu cầu, chia task, tạo PR

PM là người thao tác. Các thành viên khác xem màn hình của PM và trả lời câu hỏi.

1. **Tạo branch (1 phút)** `feature/requirements`

2. `/requirements (chủ đề)` **(7 phút)**

   Ghi "những điều cần xác nhận với khách hàng" ở chương 2 vào mục "Làm gì"

3. `/task-breakdown` **(3 phút)**

   Chia **ít nhất bằng số người, tối đa 10 task**. Không ghi người phụ trách ở bước này

4. **Ghi "thao tác trình bày" vào README.md rồi tạo PR (2 phút)**

   Commit 3 file (requirements / tasks / README)

Chung cho 5 chủ đề: dữ liệu lưu trong trình duyệt (`localStorage`), không dùng server. Không dùng ảnh, chỉ hiển thị bằng emoji và màu.

CHƯƠNG 5 • Mọi người trừ PM (2 phút)

Cả nhóm kiểm tra PR yêu cầu

Nội dung cần kiểm tra (Files changed)

- Chức năng mình định làm có nằm trong mục “không làm gì” không
- Mục “không làm gì” còn thiếu điều gì không

Sau khi mọi người đã xem

1 người Approve rồi merge.

Sau đó **mọi người cập nhật main** (các bước giống chương 4).

> Khi cả nhóm đã đọc yêu cầu trong PR, sau này sẽ không có trường hợp “tôi chưa được nghe”.

GÓC MỞ RỘNG • Thuật ngữ về Issue

Thuật ngữ về Issue

Từ	Nghĩa
Issue	"Việc cần làm" được đăng ký trên GitHub. Hôm nay, mỗi task là một Issue
Người phụ trách (Assignee)	Người thực hiện Issue đó. Tên hiện trong danh sách Issue
Issue đang mở / đã đóng	Issue chưa xong / Issue đã xong
Closes #số	Nếu ghi vào phần mô tả PR, khi PR được merge vào main thì Issue có số đó sẽ tự đóng
Số Issue	Issue và PR dùng chung dãy số. Đã có 2 PR thì Issue tạo tiếp theo là #3

CHƯƠNG 5 • PM (2 phút)

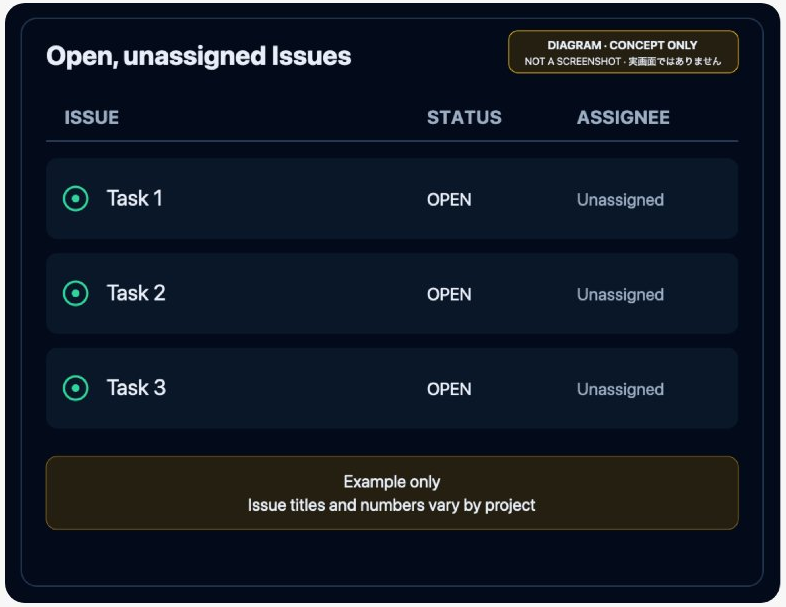
Chuyển task thành Issue

Sau khi merge PR yêu cầu, hãy gửi đoạn dưới trong chat mới.

```
/create-issues
```

> Mỗi task sẽ được tạo thành một Issue. **Chưa gán người phụ trách (Assignee).** Mỗi người tự gán mình bằng `/start-task` khi bắt đầu. Dù gửi 2 lần, Issue cũng không bị tạo trùng.

Cả nhóm mở tab **Issues** trên GitHub và kiểm tra đã có đủ Issue bằng số task.



CHƯƠNG 6 • Kiểm tra PR

Cần kiểm tra gì trước khi Approve

	Buổi 3 (trước khi Keep)	Hôm nay (trước khi Approve)
Đối tượng xem	diff	**Files changed** của PR
Nội dung kiểm tra	Đã thỏa điều kiện hoàn thành chưa	**Có chạy trên trình duyệt đúng điều kiện hoàn thành không**
Tiêu chí	Điều kiện hoàn thành trong `tasks.md`	**Giống vậy** (`docs/tasks.md`)

> **Không cần đánh giá code tốt hay xấu.** Chỉ cần kiểm tra 2 điều: ① có chạy trên trình duyệt đúng điều kiện hoàn thành không ② có tính năng nào không được yêu cầu mà vẫn bị thêm không.

CHƯƠNG 6 • Cách bắt đầu task

Bắt đầu task bằng /start-task

Việc /start-task thực hiện	Lý do
Liệt kê các Issue chưa có người phụ trách	Biết được task nào còn trống
Gán mình làm người phụ trách (Assignee) của Issue đã chọn	**Người khác không bắt đầu cùng task.** Issue đã có người thì không chọn được
Cập nhật main **rồi tạo branch từ đó**	Nếu tạo từ main cũ, sau này PR sẽ bị xung đột (conflict)
Hiện "làm gì" và "điều kiện hoàn thành" của task	Biết được tiếp theo cần gửi gì

> Trong mô tả PR, hãy ghi `Closes #số` (số của Issue). Khi PR được merge vào main, Issue sẽ tự đóng.

CHƯƠNG 6 • Engineer (8 phút)

Triển khai task 1

1. **Trong chat mới: `/start-task` → chọn Task 1**

   Bạn trở thành người phụ trách và branch `feature/<số>-...` được tạo

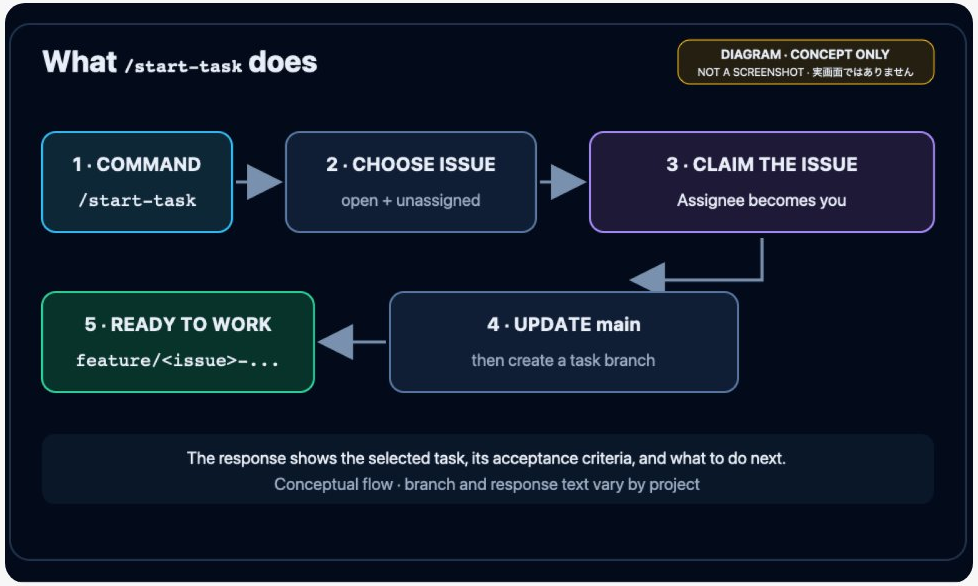
2. **Trong chat mới: chỉ gửi phần "làm gì"**

   Điều kiện hoàn thành và "đừng đổi chức năng khác" đã được ghi trong file rule

3. **Xem diff → kiểm tra điều kiện hoàn thành trên trình duyệt → Keep**

4. **Commit → PR. Trong mô tả ghi `Closes #số`**

   Khi nhờ Agent, hãy thêm câu "ghi Closes #số vào mô tả"



Trong lúc chờ, các thành viên khác đọc điều kiện hoàn thành của các task khác ở tab Issues. Chỉ chạy `/start-task` sau khi task 1 đã được merge.

CHƯƠNG 6 • QA (4 phút)

Kiểm tra task 1 rồi merge

1. **Tên branch ở góc dưới bên trái → branch của Engineer**

   Chọn `origin/feature/<số>-...` để lấy về máy mình

2. **Nhấp phải vào file HTML → Open In Browser**

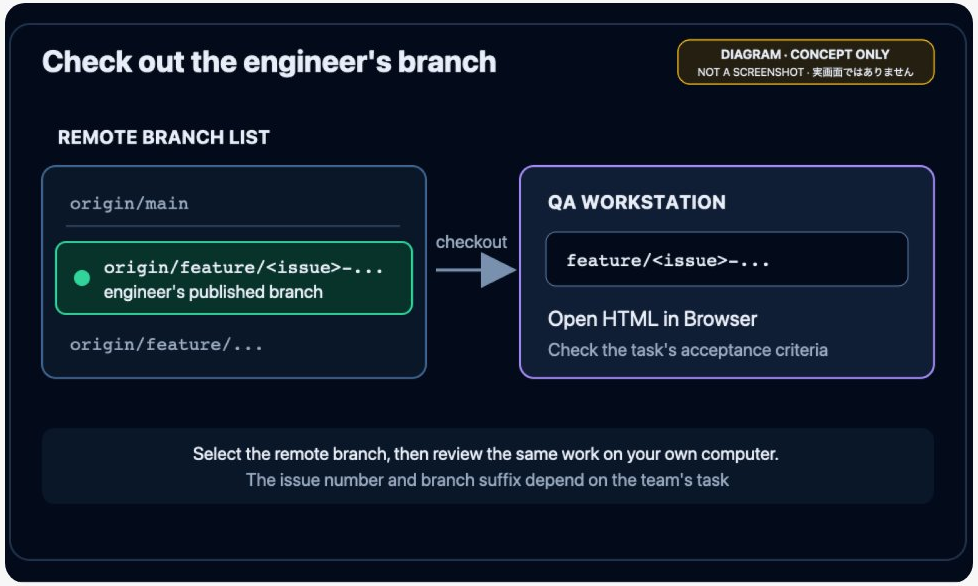
   Kiểm tra có chạy đúng **điều kiện hoàn thành của task 1** không

3. **Xem Files changed của PR**

   Kiểm tra không có file nào không được nhờ bị thêm vào

4. **Approve → Merge pull request**

   Issue của task 1 sẽ tự đóng



CHƯƠNG 6 • Mọi người trừ Engineer

Bắt đầu task mình phụ trách

1 Trong chat mới: `/start-task` → chọn một Issue còn trống (3 phút)

Bạn trở thành người phụ trách và branch được tạo từ `main` mới nhất. **Issue người khác đã lấy thì không chọn được**

2 Triển khai task của mình (nếu còn thời gian)

Các bước giống task 1. Trong mô tả PR ghi `Closes #số`. **Tạo được PR là đạt mục tiêu 4 của hôm nay**

Nếu `/start-task` báo "không dùng được `gh`", hãy làm thủ công: ở Issue, nhấn **Assignees** bên phải để chọn mình → `main` → Sync Changes → Create new branch...

CHƯƠNG 6 • Mức hoàn thành

Mức hoàn thành hôm nay

- [ ] PR của task 1 đã được merge
- [ ] Trên `main` ở máy mọi người, **màn hình của task 1 hiện trên trình duyệt**
- [ ] Mọi người đã nhận 1 Issue mình phụ trách bằng `/start-task`
- [ ] **Đã tạo PR cho task mình phụ trách** THỬ

Hoàn thành đến mục 2 là đủ. Mục 3 và 4 có thể làm tiếp ở nửa đầu buổi 5.

CHƯƠNG 6 • Khi gặp vấn đề

Cách xử lý khi bị conflict

Khi PR hiện **This branch has conflicts**, hãy gửi đoạn dưới cho Agent.

```
Lấy bản mới nhất của main vào branch hiện tại.
Nếu có conflict thì đề xuất cách giải quyết, giữ lại thay đổi của cả hai bên.
Giải quyết xong thì commit, nhưng chưa push.
```

> Nếu máy của mình hiện màn hình conflict, hãy nhấn **Resolve in Chat**. Agent sẽ xem thay đổi của cả hai bên và đề xuất cách sửa.

Cách sửa đề xuất cũng phải xem diff rồi mới nhấn Keep.

CHƯƠNG 7 • Báo cáo tiến độ (mỗi nhóm 30 giây)

3 nội dung PM báo cáo

01 Chủ đề và thao tác trình bày

02 Số Issue đã đóng (số task đã xong)

03 Việc đầu tiên sẽ làm ở buổi sau

CHƯƠNG 7 • Tổng kết hôm nay

Điều cần nhớ hôm nay

01

Chia sẻ thay đổi code bằng PR

Người tạo PR và người merge là hai người khác nhau

02

Task thành Issue, bắt đầu bằng `/start-task`

Biết được ai đang làm gì và không có trường hợp 2 người làm cùng một task

03

Kiểm tra PR bằng điều kiện hoàn thành

Không xét code tốt hay xấu, mà xem có chạy trên trình duyệt đúng điều kiện hoàn thành không

04

Rule và yêu cầu cũng chia sẻ được bằng PR

File do 1 người thêm vào thì cả nhóm đều dùng được

CHƯƠNG 7 • Buổi sau

Buổi 5 và bài tập về nhà

	Nội dung buổi 5
Nửa đầu	Hoàn thành các task còn lại (**không thêm chức năng mới**)
Giữa buổi	Làm cho thao tác trình bày chạy được **trên branch** main
Nửa sau	Cả nhóm trình bày trong 5 phút

> **Bài tập (không bắt buộc):** làm tiếp task của mình và tạo PR. Khi xong, hãy nhận Issue còn trống bằng `/start-task`. Trước khi bắt đầu, luôn cập nhật main.

Khi trình bày sẽ chạy ứng dụng trên branch `main`. Những thay đổi chưa được merge thì không thể dùng để trình bày.